

Lincoln University College

Faculty of Computer Science and Multimedia

**PocketDRS: A Single-View 3D Trajectory
Reconstruction and Decision Review System for
Cricket**

A PROJECT REPORT

Submitted to

Department of Information Technology

Phoenix College of Management

Maitidevi, Kathmandu

In partial fulfilment of the requirements for the Bachelor in Information
Technology (BIT)

Submitted by

Niraj Kafle

BIT 7th Semester

University ID : LC0003001674

2026/05/30

2. A cricket ball is small, fast, and motion-blurred, and can easily be confused with background objects, players, shadows, or spectators.
3. Small errors in marking the pitch corners produce much larger errors at the far end of the pitch, because the far end occupies only a few pixels.
4. Lighting changes, camera shake, the bowler or batter blocking the view, and inconsistent recording conditions all make a real ground less predictable than a controlled broadcast environment.

Therefore, the central problem addressed by this project is whether a single-phone-camera workflow, supported by pitch calibration and lightweight computer-vision techniques, can produce a sufficiently consistent and understandable LBW review aid for ordinary cricket grounds.

1.3 Objectives

1.3.1 General Objective

To design and implement a low-cost cricket decision-support system that uses a single camera view to analyse a delivery, estimate the ball path in calibrated pitch coordinates, and assist LBW review.

1.3.2 Specific Objectives

1. To develop a user-guided calibration process that marks the pitch corners and stumps to recover the camera pose.
2. To detect and track the ball from a single-camera delivery video and reconstruct its path in real-world 3D using physics-based fitting.
3. To estimate an LBW outcome using rule-based checks on the pitching line, impact line, and projected stump contact.
4. To present the result through an interactive 3D visualisation that helps users interpret the decision.

1.4 Scope and Limitation

1.4.1 Scope

The PocketDRS project encompasses the complete workflow from video capture to LBW decision:

- Mobile application for video selection, trimming, and pitch calibration.
- Backend processing pipeline for ball detection, tracking, and 3D reconstruction.
- LBW decision engine implementing ICC Rule 36.
- Interactive 3D visualisation of the trajectory.
- Local and cloud-based persistent storage of pitches and analyses.

1.4.2 Limitation

- The system operates under umpire-POV camera placement; other angles may require retraining or recalibration.
- Accuracy degrades with heavy ball occlusion, poor lighting, or unstable camera mounting.
- The system is not ICC-certified and is designed for decision support rather than final authority.
- Monocular depth estimation remains fundamentally limited compared to multi-camera triangulation.
- Performance depends on proper pitch calibration; user error in marking reference points reduces decision reliability.

1.5 Development Methodology

PocketDRS followed an incremental, agile approach: requirement analysis (from literature and existing systems), proof-of-concept on calibration / detection / 3D reconstruction, iterative development of the full pipeline with continuous testing, unit and synthetic-harness validation, and deployment of the backend on Google Cloud Run with Firestore persistence alongside the mobile app on Android and iOS.

1.6 Report Organisation

The report is organised into five chapters:

- **Chapter 1: Introduction** introduces the project, the problem statement, the objectives, the scope and limitations, the development methodology, and the organisation of the report.

- **Chapter 2: Background Study and Literature Review** presents the underlying theory (camera geometry, projectile motion, RANSAC, ICC Rule 36) and reviews the related work.
- **Chapter 3: System Analysis and Design** covers the functional and non-functional requirements, feasibility analysis, ER model, data flow diagrams, system architecture, and the five-step algorithm.
- **Chapter 4: Implementation and Testing** details the implementation of the modules, the unit and system tests, the synthetic validation, and the real-world clips.
- **Chapter 5: Conclusion and Future Recommendations** summarises the outcome, the lessons learnt, and the recommendations for future work.

The references and the appendices follow Chapter 5.

Chapter 3:

SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

3.1.1 Requirement Analysis

Functional Requirements

Table 3.1: Functional requirements of the PocketDRS system.

Requirement	Description
User Authentication	Users sign in via Firebase Authentication with email/-password or social login.
Video Capture/Selection	Users can record a new video or select an existing video from the gallery.
Video Trimming	Users trim the selected video to isolate the delivery of interest.
Pitch Calibration	Users calibrate by marking the pitch corners and both sets of stumps (base and top); the camera pose is anchored on the stump marks.
Calibration Validation	The system validates calibration quality and alerts users if quality is poor.
Save Pitch Config	Calibration data is saved locally and to Firestore for reuse on the same ground.
Job Submission	The user submits the trimmed video and calibration for server-side processing.
Ball Detection	The server detects ball candidates using motion and colour cues.
3D Reconstruction	The server recovers the ball trajectory in world coordinates using physics fitting.
LBW Decision	The server evaluates and returns a decision (OUT / NOT OUT / UMPIRE'S CALL) with rationale.
3D Visualisation	The client displays an interactive 3D trajectory using Three.js in a WebView.
Decision History	Users view past analyses and decisions saved to Firestore.

The table above specifies the functional requirements of the system. As the project follows the structured approach, these requirements are modelled in detail by the data flow diagrams that follow, rather than by the object-oriented class, sequence, state, or deployment diagrams.

Non-functional Requirements

Table 3.2: Non-functional requirements of the PocketDRS system.

ID	Requirement	Description
NFR-01	Response Time	Processing should complete within 30 seconds for a 5-second delivery clip.
NFR-02	Accuracy	Reconstructed impact position within ± 110 cm and bounce within ± 175 cm of ground truth on the synthetic harness (honest monocular bounds); structural correctness on real handheld video confirmed.
NFR-03	Offline Support	Pitch calibration and video selection work offline; analysis requires connectivity.
NFR-04	Security	User authentication and data encryption in transit; Firebase rules enforce privacy.
NFR-05	Cross-Platform	App runs on Android 6.0+ and iOS 12.0+ via Flutter.
NFR-06	Usability	First-time users can complete a full analysis within 5 minutes with minimal guidance.
NFR-07	Maintainability	Code is modular and documented to support future enhancements.
NFR-08	Data Privacy	User data is not shared with third parties; deletion requests are honoured.

3.1.2 Feasibility Analysis

Technical Feasibility

The system uses well-known, mature tools. Flutter handles the mobile app on Android

and iOS. FastAPI runs the backend. OpenCV, NumPy, and SciPy do the computer-vision work. Firebase handles sign-in and cloud storage. A working prototype already runs the full upload-to-result flow on ordinary phones, so the stack is proven.

Operational Feasibility

The workflow is short and repeatable. The user places a phone behind the bowler-end stumps, records the delivery, taps four pitch corners to calibrate, trims the clip, and waits for the result. No training is needed beyond a short on-screen guide. It fits coaching nets, schools, and clubs.

Economic Feasibility

The cost is essentially the phone and a tripod. There are no camera arrays, no licensed broadcast hardware, and no paid SDKs; the software is open source and the cloud runs on a free-tier-friendly Cloud Run service. A professional DRS install runs into hundreds of thousands of dollars; PocketDRS runs for the price of any modern smartphone.

Schedule Feasibility

The project fits one semester. Requirements and design took weeks 1–3, the pipeline and app took weeks 4–12, and validation and the report took the final three weeks. Each module was built and tested on its own before being wired together, which kept slippage local.

3.1.3 Data Modelling: ER Diagram

The data model has six entities. A *User* (Firebase UID) owns many *Pitches* and submits many *Analyses*. Each *Pitch* has one *CalibrationProfile* (the saved corner and stump taps). Each *Analysis* uses one *Pitch*, holds one *DeliveryVideo* (the trimmed clip), and produces one *LBWResult*. Figure 3.1 shows the full diagram with attributes and crow’s-foot cardinalities.

In Firestore the same model lives under per-user collections: `users/{uid}` for the user document, `users/{uid}/pitches/{pitchId}` for saved calibrations, and `users/{uid}/analyses/{analysisId}` for completed analyses.

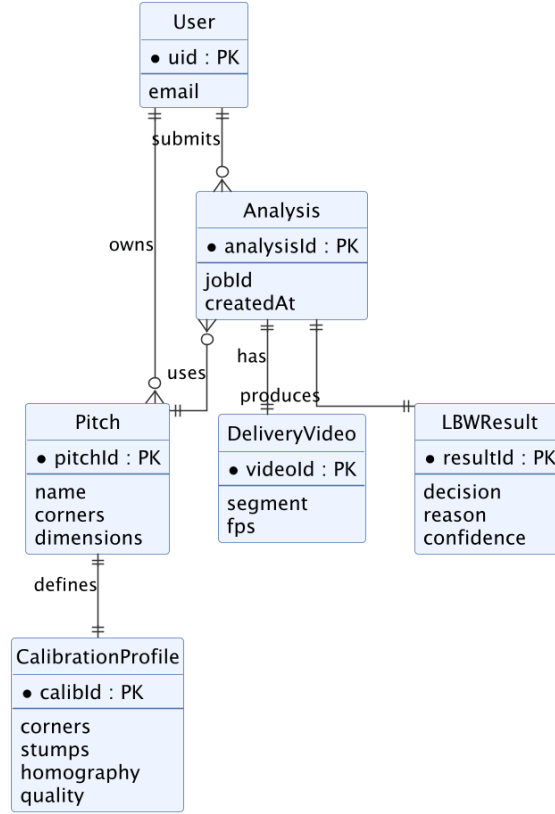


Figure 3.1: PocketDRS entity–relationship diagram.

3.1.4 Process Modelling: Data Flow Diagram (DFD)

The Level 0 DFD treats the whole system as one process. The User sends the clip and the calibration taps; Firebase exchanges sign-in tokens and stores the result; the system returns the LBW decision and the 3D view (Figure 3.2).



Figure 3.2: Level 0 (context) DFD.

The Level 1 DFD splits the system into four processes: (1) Capture & Trim Delivery on the mobile app, (2) Calibrate Pitch (collects corner and stump taps), (3) Analyse Delivery on the server (decode, detect ball, track, rebuild 3D, decide LBW), and (4) Present Result (3D view and verdict). Two data stores connect them: the Calibration Store (D1) and the Job/Result Store (D2). See Figure 3.3.

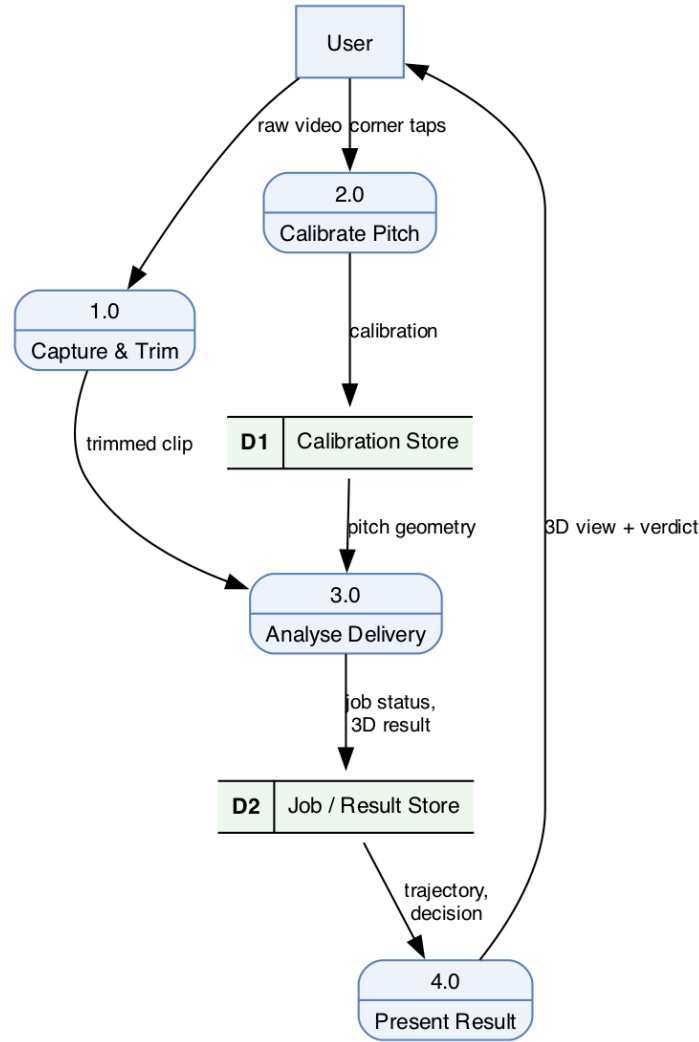


Figure 3.3: Level 1 DFD.

3.2 System Design

3.2.1 Architectural Design

PocketDRS follows a three-tier design: a client tier, a server tier (the API and the processing pipeline), and a storage tier. Figure 3.4 shows how they connect.

Client. The Flutter app on the phone handles sign-in, video recording or picking, trimming, pitch-corner tapping, job submission, and the embedded Three.js result viewer.

API. A FastAPI service exposes REST endpoints to upload a job, poll its status, and fetch the result. Uploads return immediately; the heavy work is queued and runs in the background so HTTP calls never block.

Pipeline. The pipeline is the computer-vision core (OpenCV, NumPy, SciPy, optional YOLO). It decodes the video, finds the ball, builds the trajectory, lifts it to 3D, and

applies the LBW rule. It writes progress to Firestore as it goes.

Storage. Firebase Authentication holds user identity. Firestore stores user, pitch, analysis, and live job documents. Cloud Storage holds the uploaded video files. The whole pipeline runs on Cloud Run, which scales the container up on demand and down to zero when idle.

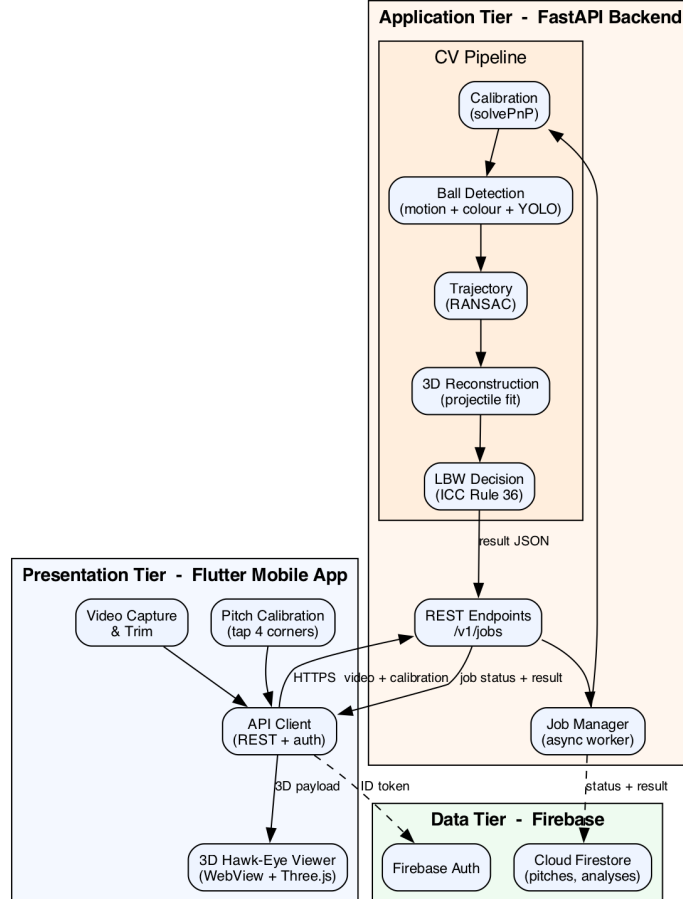


Figure 3.4: Three-tier system architecture of PocketDRS.

3.2.2 Database Schema Design

Firestore uses four collections, summarised in Table 3.3.

Table 3.3: Firestore collections used by PocketDRS.

Collection path	What it holds
<code>users/{uid}</code>	User document: email and profile fields.
<code>users/{uid}/ pitches/{pitchId}</code>	Saved calibration: corners, stumps, homography, quality score, timestamp.
<code>users/{uid}/ analyses/{analysisId}</code>	Finished analysis: video metadata, calibration used, 2D track, 3D path, LBW verdict, warnings.
<code>job_queue/{jobId}</code>	Live job state: status, progress, error. Deleted once the result is saved.

3.2.3 Interface Design (UI/UX)

The app has seven screens: *Sign In*, *Home* (history of past analyses), *Calibration* (tap the four pitch corners with live quality feedback), *Video Selection / Trim*, *Progress*, *Result*, and *Settings*. The Result screen is the main output surface. It embeds the Three.js viewer in a WebView and shows the recovered 3D ball path with the LBW verdict; the user can orbit, pan, and zoom freely.

Figure 3.5 shows three renders from the actual viewer (`hawkeye.js`), one for each verdict class (OUT, NOT OUT, UMPIRE’S CALL). The blue line is the pre-bounce path, the red dot is the bounce, the green line goes from bounce to pad, and the orange line is the predicted continuation onto the stumps.



Figure 3.5: Result screen for OUT (left), NOT OUT (middle), UMPIRE'S CALL (right).

3.2.4 Physical DFD

The physical DFD maps the logical processes onto the real, deployed infrastructure: phone, Cloud Run service, Firestore, and Cloud Storage. Figure 3.6 shows the layout.

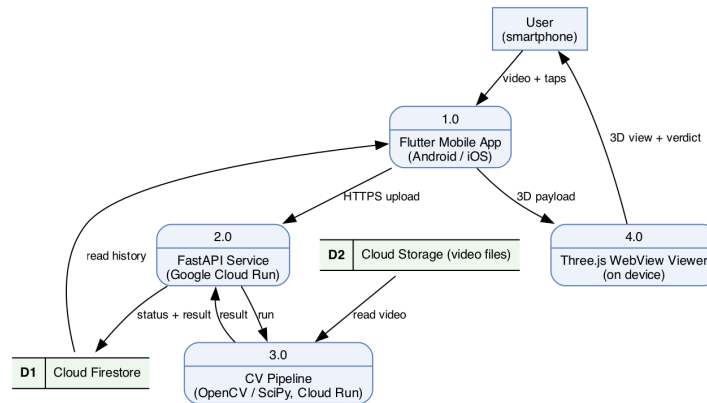


Figure 3.6: Physical DFD of the deployed system.

3.3 Algorithm Details

The server analysis runs in five steps, one after the other. Each step takes a small, named input, produces a small, named output, and is described below. The maths

behind each step is standard computer vision and is cited in the references.

3.3.1 Step 1: Finding Where the Camera Was Standing

Given: the pitch corners and the two sets of stumps (base and top) the user marked on the first frame, and the known real-world geometry of a cricket wicket (stump width and height).

Produces: the camera's position and the direction it was pointing.

Because a cricket pitch is always the same size (about 20 m long and 3 m wide), the system can compare the shape of the pitch as it appears in the image with its known real shape. From the difference between the two, it works out where the camera must have been standing and which way it was facing. This is a well-known technique in computer vision. One known problem is that the maths has two possible answers, one with the camera above the ground and one mirrored below it; the system simply rejects the below-ground answer, since a real camera is always above the pitch.

3.3.2 Step 2: Finding the Ball in Each Frame

Given: each video frame, and the user-selected ball colour.

Produces: a list of candidate ball positions in each frame, each with a confidence score.

The ball is found in two independent ways and the results are combined:

1. **By motion.** The system compares each frame with a model of the still background and keeps only the parts that are moving. It then looks for ball-shaped moving objects. Importantly, it accepts the ball in two forms: as a small *round blob* (a slow or distant ball), and as a short *blurred streak* (a fast ball). A fast cricket ball filmed at 30 frames per second smears into a streak rather than a clean circle, so looking only for circles would miss it.
2. **By colour.** The system also looks for regions matching the ball's colour (for example, the red of a cricket ball).

When a motion detection and a colour detection agree on the same spot, the system is more confident that it has found the real ball.

Ignoring fixed clutter. Before the next step, the system removes any detection that keeps appearing in the *same* place throughout the clip. A real ball only passes through any given spot once or twice; a red or round object that sits still the whole time (a logo, a helmet, a bag, a bat handle) is not the ball. Any candidate that appears in more

than 30% of the frames at the same location is treated as background and discarded. This is the single most important fix for real-world handheld videos: without it, the next step would draw a false “ball path” through a cluster of fixed clutter.

3.3.3 Step 3: Joining the Detections into One Ball Path

Given: the list of candidate ball positions from Step 2.

Produces: a single smooth path through the frames, plus a count of how many detections fit it and how well.

Step 2 returns the real ball plus some false alarms. Step 3 finds the one consistent path that runs through them. It picks a pair of early detections as a guess, draws the smooth curve a real ball would follow from that pair, and counts how many later detections sit on that curve. The guess that explains the most detections wins.

Two small design choices make this work for phone video shot from behind the stumps. First, a ball flying *towards* the camera barely moves sideways on screen, so the inlier test uses total motion across the whole clip rather than frame-to-frame speed. Second, if the best path the search finds covers only a tiny slice of the image, it is treated as leftover clutter; the system then reports “no clear ball trajectory” instead of guessing.

3.3.4 Step 4: Rebuilding the Ball Path in Real-World 3D

Given: the ball path on the image (Step 3) and the camera position (Step 1).

Produces: the ball’s path in real-world 3D coordinates (in metres).

A single camera cannot directly see depth, so the system has to estimate how far away the ball is in each frame from how big it looks: a real cricket ball is always 36 mm in radius, so when the ball looks smaller in the image, it must be further from the camera. Combining that distance with the camera position gives a rough 3D point for each frame. These rough points are then fitted to the physics of a thrown object: a ball in flight follows a smooth curve shaped by gravity. Fitting this smooth physical curve through the rough points cancels out most of the measurement noise and gives the final 3D path. The fit is also anchored using the moment where the ball touches the ground: the bounce is the most reliable single point to pin down because, at that instant, the ball’s height above the pitch is known to be zero.

3.3.5 Step 5: Making the LBW Decision (ICC Rule 36)

Given: the 3D bounce point, the 3D point where the ball hit the pad, and the predicted point where the ball would have reached the stumps.

Produces: a decision (OUT, NOT OUT, or UMPIRE’S CALL) with a plain-language reason and a confidence score.

The decision follows the three standard LBW checks from the Laws of Cricket. It uses three known measurements: stump width, ball radius (0.036 m), and an umpire’s-call margin of one ball radius (0.036 m) horizontally and one ball diameter (0.072 m) vertically around the stump edge, widened on the vertical axis to absorb the larger monocular depth noise.

1. **Did it pitch in line?** If the ball bounced outside the line of leg stump, the verdict is NOT OUT (“pitched outside leg”).
2. **Did it hit the pad in line?** If the pad was struck outside the line of the stumps, the verdict is NOT OUT.
3. **Would it have hit the stumps?** The predicted path is carried on to the stumps. If it clearly misses, the verdict is NOT OUT (“missing stumps”).

If all three checks pass, the ball would have hit the stumps. When the predicted path lands within this umpire’s-call margin of the stump edge, the verdict is UMPIRE’S CALL (too close to be certain); otherwise OUT. The confidence score is higher when the 3D fit was clean and lower when it was noisy.